



OOP in PHP

Programmazione

PHP (acronimo ricorsivo di PHP: Hypertext Preprocessor) è un linguaggio di scripting lato server progettato principalmente per lo sviluppo di applicazioni web dinamiche.

Viene eseguito sul server e consente di generare contenuti HTML in modo dinamico, gestire dati provenienti da form, interagire con database (come MySQL o PostgreSQL), gestire sessioni e autenticazioni.

Grazie alla sua sintassi semplice, alla vasta diffusione nei contesti web e all'ampio ecosistema di framework e CMS (come WordPress), PHP è uno dei linguaggi più utilizzati per la realizzazione di siti e applicazioni web backend.



- **Lato server (Server-side scripting)**

PHP viene eseguito sul server e non nel browser dell'utente. Questo significa che il codice PHP non è visibile al client e viene usato per generare dinamicamente l'HTML finale, garantendo maggiore sicurezza e controllo della logica applicativa.

- **Integrazione nativa con HTML**

PHP è progettato per essere facilmente integrato all'interno delle pagine HTML. Questo permette di mescolare markup e logica applicativa in modo diretto, rendendo semplice la creazione di pagine web dinamiche basate su dati e condizioni.



- **Interazione con database**

Una delle forze principali di PHP è la sua capacità di comunicare in modo efficiente con i database relazionali (come MySQL, MariaDB, PostgreSQL). Consente operazioni CRUD complete ed è alla base di applicazioni data-driven come CMS, e-commerce e sistemi gestionali.

- **Ampio ecosistema e portabilità**

PHP è open source, multiplatforma e supportato da un vastissimo ecosistema di librerie, framework e CMS. Può essere eseguito su diversi sistemi operativi e server web, rendendolo una scelta flessibile e ampiamente adottata nello sviluppo web backend.



Per configurare PHP in modo semplice è consigliabile utilizzare un ambiente di sviluppo già pronto come XAMPP, WAMP o MAMP, che includono PHP, un server web (Apache) e un database.

Dopo aver installato il pacchetto, è sufficiente avviare il server web dal pannello di controllo e posizionare i file PHP nella cartella dedicata (ad esempio htdocs).

La configurazione principale di PHP avviene tramite il file php.ini, dove è possibile impostare parametri come la gestione degli errori, il fuso orario o le estensioni abilitate; una volta salvate le modifiche, basta riavviare il server per renderle effettive.



A questo punto PHP è pronto per essere utilizzato per sviluppare ed eseguire applicazioni web dinamiche in locale.

Il codice PHP è racchiuso tra i tag `<?php` e `?>`, che indicano al server dove inizia e finisce lo script.

L'istruzione `echo` serve per inviare output al browser: in questo caso stampa la stringa di testo `Hello World!` quando la pagina viene eseguita dal server.

1. `<?php`
2. `// Stampa un messaggio nel browser`
3. `echo "Hello World!";`
4. `?>`



In questo esempio viene dichiarata una variabile \$eta.

Con la struttura if / else PHP verifica una condizione logica e, in base al risultato, utilizza echo per stampare un messaggio diverso nel browser.

È un tipico controllo lato server usato per validare dati o gestire permessi.

```
1.<?php
2.$eta = 20;
3.
4.// Controllo con if
5.if ($eta >= 18) {
6.    echo "Sei maggiorenne";
7.} else {
8.    echo "Sei minorenne";
9.}
10.?>
```



Qui viene creato un array di nomi e una funzione personalizzata.

**Il ciclo foreach serve per scorrere tutti gli elementi dell'array, mentre echo stampa ogni valore andando a capo con
.**

La funzione incapsula la logica, rendendo il codice più ordinato e riutilizzabile, pratica fondamentale nello sviluppo in PHP.

```
1.<?php
2.$nomi = ["Anna", "Luca", "Marco"];
3.
4.// Definizione di una funzione
5.function stampaNomi($lista) {
6.    foreach ($lista as $nome) {
7.        echo $nome . "<br>";
8.    }
9.}
10.
11.// Chiamata della funzione
12.stampaNomi($nomi);
13.?>
```



Il codice inizia con i tag PHP `<?php ?>`, che indicano al server di interpretare il contenuto come script PHP. Viene dichiarata una variabile `$eta` usando il simbolo `$` e ogni istruzione termina con il punto e virgola ;

La funzione `isset()` viene usata per verificare l'esistenza della variabile, evitando errori. Con la struttura `if / else` viene gestita una logica condizionale che stampa un messaggio tramite `echo`.

Successivamente viene creato un array, che viene elaborato con un ciclo `foreach` all'interno di una funzione, dimostrando l'uso dei blocchi di codice `{ }`. I commenti `//` rendono il codice più leggibile ma non vengono eseguiti.

Questo esempio rappresenta una base reale e completa del funzionamento di PHP lato server.



```
1.<?php
2.// Dichiarazione di una variabile
3.$eta = 22;
4.
5.// Controllo se la variabile esiste ed è valorizzata
6.if (isset($eta)) {
7.
8.    // Condizione logica
9.    if ($eta >= 18) {
10.        echo "Accesso consentito<br>";
11.    } else {
12.        echo "Accesso negato<br>";
13.    }
14.
15.    // Array di nomi
16.    $nomi = ["Anna", "Luca", "Marco"];
17.
18.    // Funzione personalizzata
19.    function stampaNomi($lista) {
20.        foreach ($lista as $nome) {
21.            echo $nome . "<br>";
22.        }
23.    }
24.
25.    // Chiamata della funzione
26.    stampaNomi($nomi);
27.}
28.?>
```

La programmazione orientata agli oggetti, o OOP (Object-Oriented Programming), è un paradigma di programmazione che organizza il codice attorno al concetto di oggetto. Un oggetto rappresenta un'entità concreta o astratta del programma, come uno studente, un prodotto, un conto bancario o un personaggio di un videogioco, e contiene sia dati che comportamenti.

I dati sono rappresentati dalle proprietà o attributi, mentre i comportamenti sono rappresentati dai metodi, cioè funzioni appartenenti all'oggetto.

L'OOP nasce per rendere il codice più ordinato, riutilizzabile e facile da mantenere, permettendo di modellare il software in modo simile alla realtà attraverso concetti come classe, oggetto, incapsulamento, ereditarietà, polimorfismo e astrazione.



Questo primo esempio mostra la struttura più semplice della programmazione ad oggetti in PHP: una classe rappresenta un modello astratto, mentre un oggetto è un'istanza concreta di quella classe, in questo caso creiamo una classe `Studente`, con alcune proprietà e un metodo che stampa le informazioni dello studente.

```

1.<?php
2.// Definizione della classe Studente
3.class Studente {
4.
5.    // Proprietà della classe
6.    public string $nome;
7.    public string $cognome;
8.    public int $eta;
9.
10.    // Metodo per stampare le informazioni dello studente
11.    public function stampaInformazioni(): void {
12.        echo "Nome: " . $this->nome . "<br>";
13.        echo "Cognome: " . $this->cognome . "<br>";
14.        echo "Età: " . $this->eta . "<br>";
15.    }
16.}
17.
18.// Creazione di un oggetto della classe Studente
19.$studente1 = new Studente();
20.
21.// Assegnazione dei valori alle proprietà
22.$studente1->nome = "Luca";
23.$studente1->cognome = "Rossi";
24.$studente1->eta = 20;
25.
26.// Chiamata del metodo sull'oggetto
27.$studente1->stampaInformazioni();
28.}>

```

In questo esempio `Studente` è la classe, mentre `$studente1` è un oggetto. La parola chiave `$this` viene usata dentro la classe per riferirsi all'oggetto corrente.

Le proprietà sono dichiarate `public`, quindi possono essere lette e modificate direttamente dall'esterno.



In questo esempio introduciamo il costruttore, cioè un metodo speciale chiamato automaticamente quando viene creato un oggetto, il costruttore permette di inizializzare subito le proprietà dell'oggetto, evitando di dover assegnare i valori manualmente dopo la creazione.

```

1.<?php
2.
3.// Definizione della classe Prodotto
4.class Prodotto {
5.    // Proprietà della classe
6.    public string $nome;
7.    public float $prezzo;
8.    public int $quantita;
9.
10.    // Costruttore della classe
11.    public function __construct(string $nome, float $prezzo, int $quantita) {
12.        $this->nome = $nome;
13.        $this->prezzo = $prezzo;
14.        $this->quantita = $quantita; }
15.
16.    // Metodo che calcola il valore totale del prodotto in magazzino
17.    public function calcolaTotale(): float {
18.        return $this->prezzo * $this->quantita; }
19.
20.    // Metodo che stampa i dati del prodotto
21.    public function stampaProdotto(): void {
22.        echo "Prodotto: " . $this->nome . "<br>";
23.        echo "Prezzo: €" . $this->prezzo . "<br>";
24.        echo "Quantità: " . $this->quantita . "<br>";
25.        echo "Totale: €" . $this->calcolaTotale() . "<br>"; }
26.
27.    // Creazione dell'oggetto passando i valori al costruttore
28.    $prodotto1 = new Prodotto("Mouse", 25.50, 4);
29.
30.    // Chiamata del metodo
31.    $prodotto1->stampaProdotto();
32.}>

```

Il metodo `__construct()` viene eseguito automaticamente nel momento in cui scriviamo `new Prodotto(...)`.

Questo rende il codice più ordinato e sicuro, perché obbliga a creare l'oggetto già con i dati necessari.



Buon Davante a tutti

